

完整软件过程定义文档

一、引言

1.1 项目背景

随着生成式人工智能和大语言模型逐渐进入软件开发活动，软件过程中的部分工作方式正在发生变化。需求整理、设计草案生成、局部代码实现、测试用例编写和日志分析等活动，过去主要依赖项目成员手工完成，现在可以在一定程度上借助 AI 工具生成初稿、提供候选方案或辅助分析结果。AI 技术的价值不在于替代完整的软件开发过程，而在于帮助团队提高部分重复性、结构化工作的效率，并为后续评审和修改提供基础材料。

本报告以“多智能体在线语音狼人杀系统”为对象，讨论 AI 技术介入后软件过程的改进方式。该系统以在线狼人杀为业务场景，涉及房间管理、玩家交互、语音或文本发言、游戏状态推进、角色技能处理、投票结算、胜负判定和 AI 玩家参与等功能。其中，AI Service 模块需要根据游戏阶段、玩家角色、可见信息和历史记忆生成 AI 玩家行为，并处理角色私有信息保护、Prompt 构造、模型结果解析、异常兜底和多 AI 狼人协同等问题。与普通业务系统相比，该项目既包含常规软件开发活动，也包含大模型输出不稳定、角色身份信息隔离和 AI 行为约束等新的过程管理问题。

因此，在该项目中引入 AI 技术，不能简单理解为让 AI 自动完成需求、设计、编码或测试工作。更合理的做法是将 AI 作为软件过程中的辅助工具，使其参与初稿生成、方案建议、局部代码辅助、测试用例生成和复盘摘要等环节。AI 输出只有经过人工审核、质量标准检查和过程记录后，才能进入正式交付物或过程资产。基于这一思路，本报告对原有软件过程进行改进设计，并结合原型验证结果，形成一套适合本项目的 AI 增强软件过程。

1.2 报告目标

本报告的目标是建立一套面向“多智能体在线语音狼人杀系统”的 AI 增强软件过程方案。该方案在保留原有需求分析、系统设计、编码实现、测试验证和交付总结等阶段的基础上，明确 AI 可以参与的活动节点、输入输出、人工审核责任和质量控制方式。

具体而言，报告主要解决三个问题：一是分析原有软件过程中哪些环节存在效率低、重复性强或容易遗漏的问题；二是说明 AI 技术如何以可控方式融入需求、设计、编码、测试和复盘等活动；三是结合 AI 辅助生成测试用例和自动化测试代码的原型验证结果，判断该改进方案在课程项目中的可行性。通过这些内容，报告最终形成改进后的软件过程定义，包括阶段活动流程、角色职责、交付物清单、质量标准和过程度量指标。

二、项目与原有软件过程概述

2.1 项目与过程背景

多智能体在线语音狼人杀系统是一个支持真人玩家与 AI 玩家同局博弈的在线游戏平台。系统围绕房间创建、角色分配、昼夜阶段推进、语音或文本发言、投票结算、技能处理和胜负判定展开，并通过 AI Service 为 AI 玩家提供角色人格、记忆管理、上下文组装、Prompt 构造、模型结果解析、阶段决策、异常兜底和多 AI 狼人协同能力。

原有软件过程采用阶段式开发方式，能够支撑课程项目从需求到交付的基本活动，但 AI 相关内容尚未进入正式过程定义。尤其在 AI 玩家模块中，需求规则、模型输出、身份信息过滤和异常兜底都需要更明确的活动节点、人工审核和质量标准。

2.2 原有过程概述与主要问题

原有阶段	主要活动	主要交付物	原有质量控制	主要问题
项目启动与计划	明确目标、范围、任务分工和进度	项目计划、任务分工	计划评审	AI 工具使用边界和记录方式不明确
需求分析	收集功能需求、非功能需求和游戏规则	SRS、用户故事、需求追踪矩阵	需求评审	需求整理耗时，验收标准和追踪关系容易不完整
系统设计	划分模块，设计接口、数据结构和核心流程	设计说明书、UML 模型、接口文档	设计评审	UML 和接口草案主要依赖人工准备
编码实现	完成模块编码、调试、提交和重构	源代码、单元测试记录	代码审查、版本控制	局部代码和异常处理重复性较高
测试验证	设计测试用例，执行功能、异常和回归测试	测试用例、测试报告、缺陷记录	测试评审、缺陷闭环	边界场景、异常路径和内部规则容易遗漏
交付与总结	整理版本、发布说明和项目复盘材料	发布说明、项目总结报告	交付检查、复盘会议	日志、缺陷和测试结果整理依赖人工

表1 原有软件过程概述与主要问题

综合来看，原有过程的问题集中在“人工起草成本高、过程资产生成慢、测试自动化转化不足、复盘材料结构化不足”。因此，AI 适合被放入输入输出明确、重复性较强、结果便于人工检查的活动节点，而不是替代阶段负责人作出需求、设计、合并或发布决策。

三、改进后的软件过程定义

3.1 过程概述与目标

改进后的软件过程仍以八阶段迭代式过程为基础，即项目启动与计划、需求获取与分析、原型设计与可行性验证、系统架构与详细设计、编码实现与单元验证、集成测试与迭代修正、交付与确认、项目总结与过程改进。该阶段划分参考软件生命周期过程、过程评估和过程度量相关标准对活动、交付物、质量控制和度量的要求。更新将在原有阶段中增加 AI 辅助节点、AI 输出审核机制、AI 过程记录和度量指标，使 AI 技术能够以可控方式融入软件过程。

在改进后的软件过程中，AI 主要承担辅助生成和辅助分析工作。例如，在需求获取与分析阶段，AI 可辅助生成需求文档初稿、用户故事、验收标准和需求追踪建议；在原型设计与可行性验证阶段，AI 可辅助整理验证目标、生成验证用例和归纳 LLM、STT/TTS、AI 行为生成等技术验证结果等。

新的软件过程定义将 AI 的使用纳入软件过程管理后，使 AI 生成内容能够被记录、被审核、被验证和被度量。AI 输出只作为中间成果，不能直接替代正式交付物。需求是否进入基线、设计是否通过评审、代码是否合并、测试是否通过、版本是否交付，仍然由对应阶段的项目成员按照质量标准进行确认。通过这种方式，可以在保持原有过程责任边界的同时，提高需求整理、设计建模、局部编码、测试设计和复盘总结等活动的效率。

3.2 软件过程阶段划分

阶段编号	阶段名称	过程目标	主要活动	AI介入方式	主要输出	阶段质量控制
S1	项目启动与计划	明确项目目标、范围、分工、进度和主要风险	分析项目背景，确定系统目标和功能边界；划分开发角色；制定进度计划；识别项目风险	AI 辅助整理项目计划模板、任务拆分建议、风险清单和 AI 工具使用规范	项目计划书、任务分工表、AI 工具使用说明	项目计划经组内评审；AI 使用边界、数据脱敏要求和记录方式明确
S2	需求获取与分析	建立需求基线，明确软件需要实现的功能和质量要求	用户角色分析、功能需求分析、接口需求分析、非功能需求分析、需求优先级划分	AI 生成 SRS 初稿、用户故事、验收标准、模糊需求提示和需求追踪建议	需求分析文档、需求跟踪矩阵、AI 需求初稿、AI 输出审核记录	需求评审通过；每条需求有来源、编号、优先级和验收标准；AI 生成内容经人工确认后才能进入需求基线
S3	原型设计与可行性验证	验证核心业务和关键技术路线，降低后续实现风险	构建房间流程原型、游戏状态机原型；验证 LLM 调用、语音 STT/TTS 接口和 AI 行为生成的可行性	AI 辅助整理原型验证目标、生成验证用例、归纳 LLM 调用结果、总结语音接口和 AI 行为生成的验证结果	核心原型、技术验证结果、AI 辅助验证记录	原型能跑通核心游戏流程；关键技术验证结果有记录；AI 给出的验证结论必须由开发人员复现和确认
S4	系统架构与详细设计	将需求和原型结果转化为可实现的技术方案	总体架构设计、功能模块设计、接口设计、数据结构设计、性能设计和部署设计	AI 生成 UML 草图、模块划分建议、接口草案、异常路径说明和备选设计方案	系统设计说明书、UML 模型、接口文档、AI 设计草案	设计评审通过；设计元素可追踪到需求；接口输入、输出和异常处理清楚；AI 设计草案经设计人员修改确认

阶段编号	阶段名称	过程目标	主要活动	AI介入方式	主要输出	阶段质量控制
S5	编码实现与单元验证	逐步实现系统功能，并验证单个模块是否可用	前后端连接、游戏逻辑实现、AI Agent 模块实现、语音模块实现、数据库实现和单元测试编写	Codex、Claude Code 或 Cursor Agent 辅助局部代码生成、重构建议、测试桩生成和自动化单元测试代码生成	源代码、单元测试报告、AI 代码采纳记录、AI 辅助单元测试代码	开发人员理解并修改 AI 生成代码；代码通过 Review、静态检查和单元测试；未经审核的 AI 代码不得提交
S6	集成测试与迭代修正	验证系统是否能够支撑完整游戏流程，并根据问题进行修正	模块集成、接口联调、功能测试、异常测试、性能测试、需求跟踪检查和缺陷修正	AI 辅助生成集成测试用例、边界场景、自动化测试脚本、失败结果摘要和缺陷定位线索	集成测试记录、缺陷记录、修正结果、自动化测试脚本、AI 测试生成记录	测试用例覆盖核心需求和异常路径；缺陷记录完整；修复后通过回归测试；AI 生成测试需经测试人员审核
S7	交付与确认	完成项目验收和系统交付	系统演示、文档提交、代码提交、功能验收和版本确认	AI 辅助整理发布说明、交付检查清单、版本变更摘要和验收材料初稿	可运行系统、交付文档、验收材料、版本说明	交付物完整；代码、文档、测试报告和版本记录一致；AI 生成交付材料经项目负责人确认
S8	项目总结与过程改进	总结项目执行过程中的问题并提出改进措施	分析需求变更、团队协作、开发进度、测试缺陷和系统不足，提出后续优化方向	AI 辅助整理日志摘要、缺陷归纳、测试结果摘要、复盘草稿和过程改进建议	项目总结、改进建议、AI 复盘摘要、过程度量记录	复盘结论有事实依据；AI 生成总结只作为草稿和线索；正式改进建议由项目负责人和团队确认

表2 改进后的软件过程阶段划分

3.3 各阶段AI参与节点

阶段	AI参与节点	AI工具选型	AI输入	AI输出	人工审核人	准入要求
S1 项目启动与计划	AI辅助整理计划草案、任务拆分和风险清单	ChatGPT、Kimi、通义千问	作业要求、项目背景、成员分工、进度安排、已有项目计划	项目计划草案、任务拆分建议、风险清单、AI使用规范草案	项目负责人、配置管理员	项目范围、人员分工、进度安排和AI使用边界明确；AI使用记录方式确定
S2 需求获取与分析	AI辅助生成需求文档初稿、用户故事、验收标准和需求追踪建议	ChatGPT、Kimi；备选通义千问、Claude	游戏规则、功能清单、用户角色说明、业务流程、原始需求材料	SRS初稿、用户故事、验收标准、模糊需求提示、需求追踪建议	需求分析人员、项目负责人	需求有明确来源、编号、优先级和验收标准；AI生成内容经人工确认后才能进入需求基线

阶段	AI参与节点	AI工具选型	AI输入	AI输出	人工审核人	准入要求
S3 原型设计与可行性验证	AI辅助设计原型验证任务、生成验证用例并归纳技术验证结果	ChatGPT、Kimi、Codex；结合项目运行日志	需求基线、核心流程说明、技术栈、LLM/STT/TTS接口说明、AI行为生成目标	原型验证用例、LLM调用验证摘要、语音接口验证摘要、AI行为生成可行性分析	开发人员、项目负责人	核心游戏流程能够跑通；关键技术验证结果可复现；AI归纳结论需由人工确认
S4 系统架构与详细设计	AI辅助生成UML草图、接口草案、模块划分建议和异常路径说明	ChatGPT + Mermaid；备选 PlantUML、通义千问 + Mermaid	需求基线、原型验证结果、技术约束、模块边界、接口约定	Mermaid/PlantUML图、模块划分建议、接口草案、数据结构建议、异常路径说明	设计人员、开发人员	设计元素可追踪到需求；接口输入、输出和异常处理清楚；设计评审通过

阶段	AI参与节点	AI工具选型	AI输入	AI输出	人工审核人	准入要求
S5 编码实现与单元测试	AI辅助局部代码生成、重构建议、测试桩和单元测试代码生成	Codex、Claude Code、Cursor Agent； 质量检查配合 SonarQube、CodeQL、pytest/JUnit	需求编号、设计文档、接口说明、相关代码上下文、编码规范、测试目标	局部代码片段、重构建议、测试桩、pytest/JUnit 单元测试脚本、代码审查提示	开发人员、代码审核人、测试人员	开发人员能够解释和修改 AI 生成代码；代码通过 Review、静态检查和单元测试后才能提交
S6 集成测试与迭代修正	AI辅助生成集成测试用例、边界场景、失败摘要和缺陷定位线索	Codex、Claude Code、Cursor Agent + pytest/JUnit/Postman； 备选 ChatGPT、Kimi	集成测试记录、接口文档、缺陷记录、失败日志、需求追踪矩阵	集成测试用例建议、异常场景清单、失败摘要、缺陷定位线索、回归测试建议	测试人员、开发人员	测试用例覆盖核心流程和异常路径；缺陷可复现、可跟踪、可关闭；根因确认必须由人工完成

阶段	AI参与节点	AI工具选型	AI输入	AI输出	人工审核人	准入要求
S7 交付与确认	AI辅助整理发布说明、版本变更摘要和交付检查清单	ChatGPT、Kimi、通义千问	测试报告、缺陷关闭记录、提交记录、配置项清单、交付材料目录	发布说明草稿、版本变更摘要、交付检查清单、验收材料草稿	项目负责人、配置管理员	代码、文档、测试报告、版本记录和验收材料一致；AI生成交付材料经人工确认
S8 项目总结与过程改进	AI辅助整理日志摘要、缺陷归纳、测试结果摘要和复盘草稿	日志文件 + LLM 摘要；备选 ELK、Grafana、AIOps 工具	日志文件、缺陷数据、测试结果、过程度量指标、团队复盘记录	日志摘要、问题归纳、复盘草稿、过程改进建议	项目负责人、全体成员	AI 结论只作为复盘线索；正式总结和改进建议必须由团队确认

表3 各阶段AI参与节点表

3.4 阶段活动流程图

1) 项目启动与计划阶段

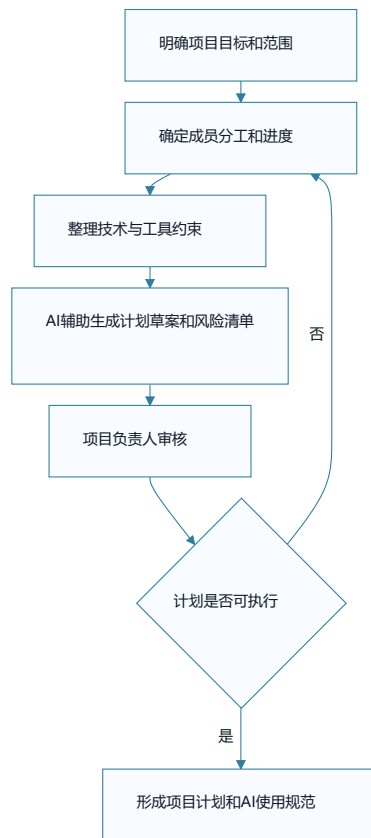


图1 S1项目启动与计划阶段流程图

2) 需求获取与分析阶段

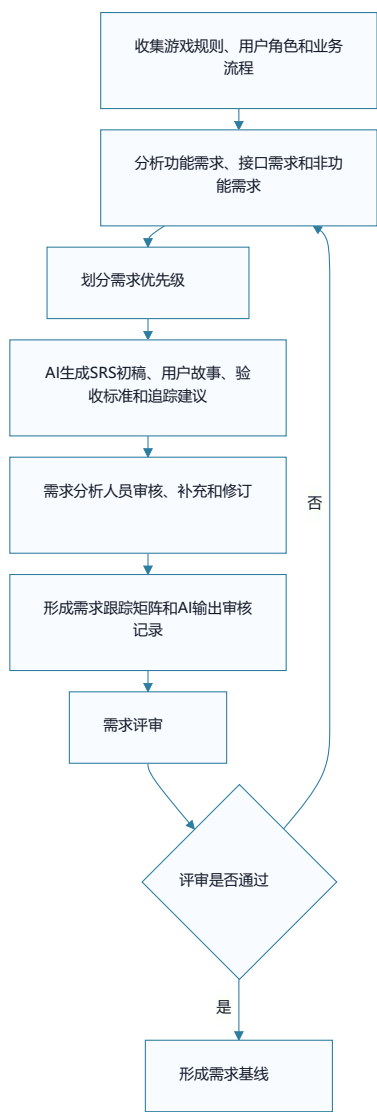


图2 S2需求获取与分析阶段流程图

3) 原型设计与可行性验证阶段

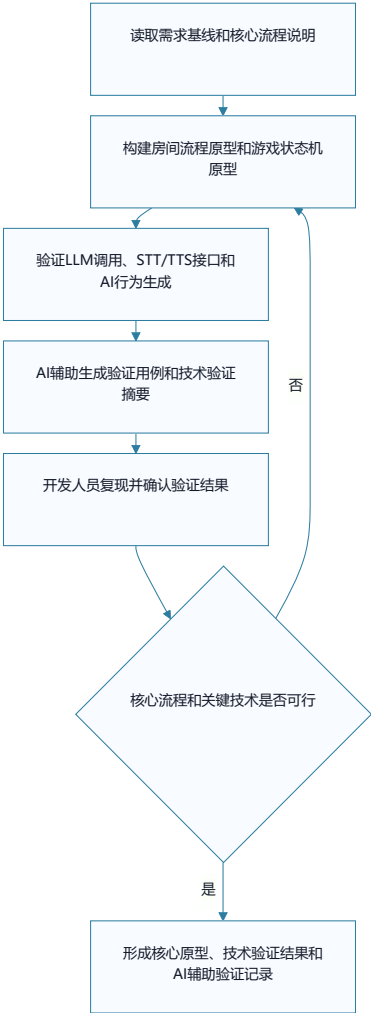


图3 S3原型设计与可行性验证阶段流程图

4) 系统架构与详细设计阶段

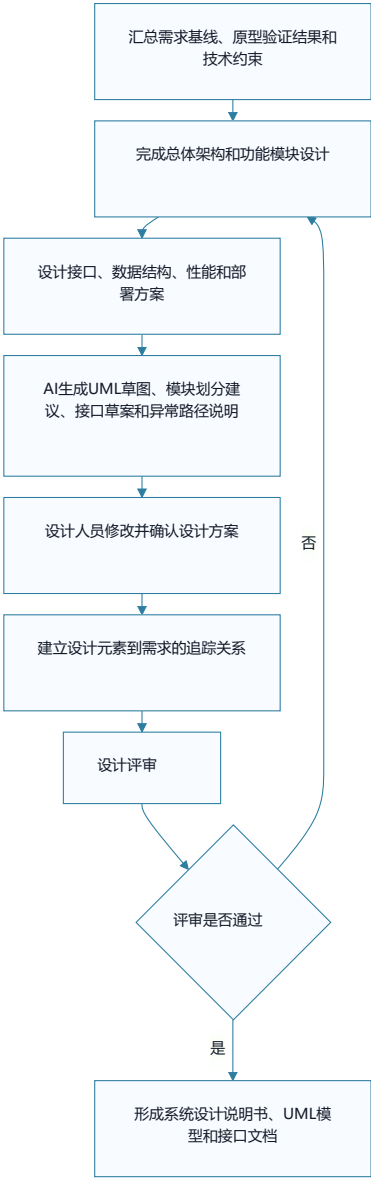


图4 S4系统架构与详细设计阶段流程图

5) 编码实现与单元验证阶段

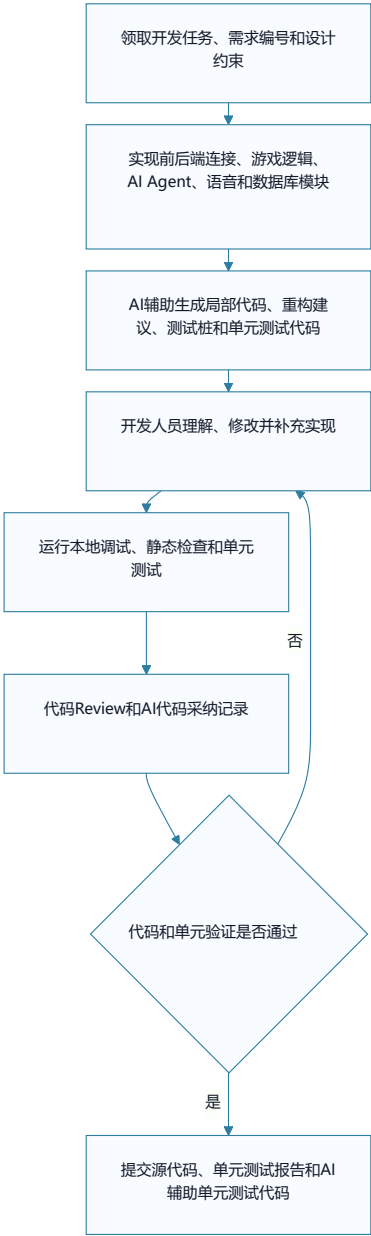


图5 S5编码实现与单元验证阶段流程图

6) 集成测试与迭代修正阶段

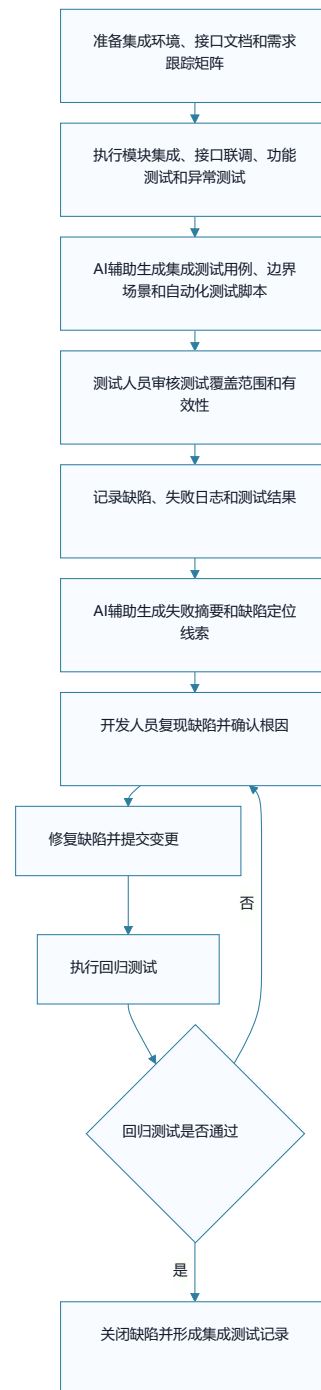


图6 S6集成测试与迭代修正阶段流程图

7) 交付与确认阶段

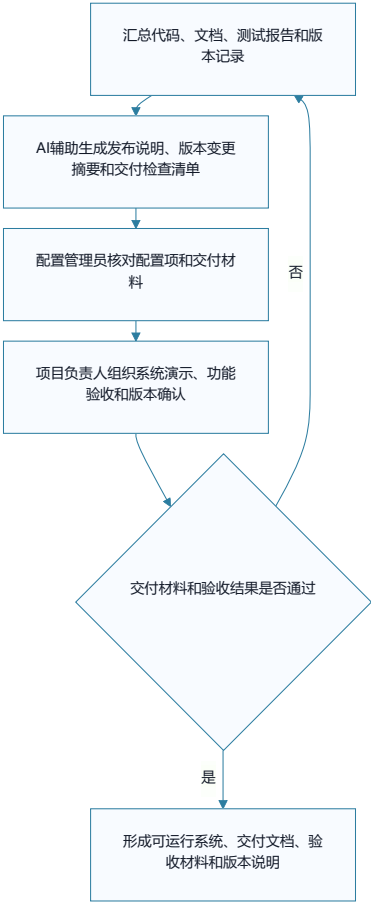


图7 S7交付与确认阶段流程图

8) 项目总结与过程改进阶段

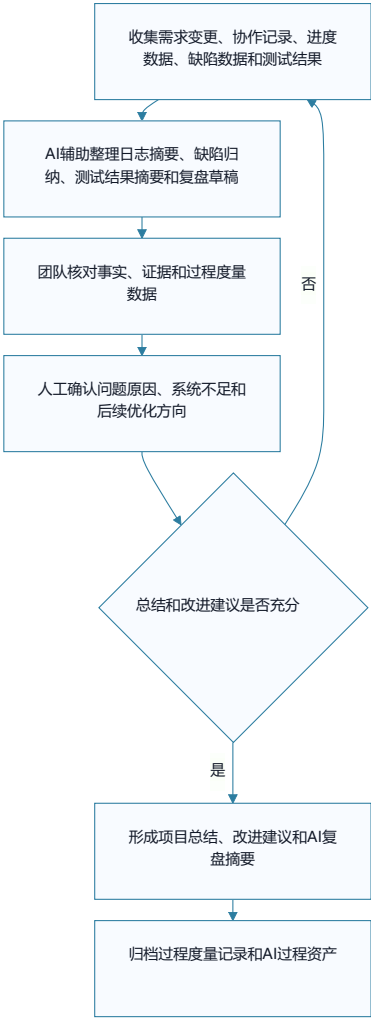


图8 S8项目总结与过程改进阶段流程图

3.5 角色职责定义

改进后的软件过程在原有项目角色基础上补充 AI 相关职责角色。AI 工具负责人、AI 输出审核人和 AI 测试分析员属于新增 AI 职责角色，可由原有成员兼任。这些角色并不替代需求分析人员、设计人员、开发人员和测试人员的原有职责，而是用于明确 AI 工具使用、AI 输出审核、AI 测试生成和过程记录等工作，使 AI 参与过程能够被管理、被追踪。

角色	类型	主要职责	参与阶段	主要交付物
AI 工具负责人	新增 AI 职责角色，可由项目负责人或配置管理员兼任	选择和维护 AI 工具，管理提示词模板，检查 AI 使用记录完整性	全过程	AI 使用记录、提示词模板库、工具使用说明
AI 输出审核人	新增 AI 职责角色，可由对应阶段负责人兼任	审核 AI 生成的需求、设计、代码、测试用例和复盘摘要是否满足阶段准入要求，记录采纳、修改或退回意见	需求获取与分析、系统架构与详细设计、编码实现与单元验证、集成测试与迭代修正、项目总结与过程改进	AI 输出审核记录、AI 采纳与修改记录
AI 测试分析员	新增 AI 职责角色，可由测试人员兼任	明确 AI 测试生成目标，设计测试生成提示词，约束测试范围和 Mock 边界，审核 AI 生成的测试用例和自动化测试代码	编码实现与单元验证、集成测试与迭代修正	测试生成提示词、测试用例表、自动化测试脚本
需求分析人员	原有角色扩展	收集和整理需求，审核 AI 生成的 SRS 初稿、用户故事、验收标准和需求追踪关系，确认需求是否进入基线	需求获取与分析	需求规格说明书、用户故事、需求追踪矩阵
设计人员	原有角色扩展	审核 AI 生成的 UML 草图、接口草案和设计建议，确认架构方案、模块边界、接口约束和异常路径是否合理	系统架构与详细设计	系统设计说明书、UML 模型、接口文档
开发人员	原有角色扩展	使用编码 Agent 辅助局部实现、重构和单元测试生成，负责理解、修改、测试并提交代码，不直接提交未经验证的 AI 代码	编码实现与单元验证	源代码、单元测试报告、AI 代码采纳记录
测试人员	原有角色扩展	评审 AI 生成的测试用例和自动化测试脚本，执行单元测试、集成测试和回归测试，记录缺陷并跟踪修复结果	编码实现与单元验证、集成测试与迭代修正	测试用例表、自动化测试脚本、测试报告

角色	类型	主要职责	参与阶段	主要交付物
项目负责人	原有角色扩展	规划 AI 使用边界，组织关键评审，检查 AI 使用记录、过程度和风险控制情况，确认项目总结和改进建议	全过程	项目计划、评审记录、项目总结报告

表4 角色职责定义表

3.6 交付物和质量标准

改进后的交付物体系仍以原有软件过程中的文档、代码、测试记录和项目总结为基础，并在此基础上增加 AI 使用记录、AI 输出审核记录、AI 采纳与修改记录等过程资产，令 AI 生成的需求、设计、代码、测试用例和复盘摘要能够被追踪和确认。

本过程对交付物质量的要求可以概括为三个方面：一是正式交付物仍需满足原有阶段质量要求，例如需求可追踪、设计可实现、代码可测试、缺陷可闭环；二是 AI 生成内容必须经过人工审核，不能直接进入需求基线、设计文档、代码库或测试资产；三是 AI 使用过程需要留下记录，包括输入材料、提示词、生成结果、审核意见和采纳状态，以便后续复盘和过程度量。

阶段	交付物	AI新增或影响内容	质量要求
S1 项目启动与计划	项目计划书、任务分工表、进度安排、风险清单	AI 辅助整理计划草案、任务拆分建议、风险清单和 AI 使用边界说明	项目范围、进度、角色分工和 AI 使用边界明确；计划经组内评审
S2 需求获取与分析	需求分析文档、用户故事、需求跟踪矩阵	AI 生成 SRS 初稿、用户故事、验收标准、模糊需求提示和追踪关系建议	需求有来源、编号、优先级和验收标准；AI 生成内容经人工确认后才能进入需求基线
S3 原型设计与可行性验证	核心原型、技术验证结果、原型评审记录	AI 辅助生成验证用例，归纳 LLM 调用、STT/TTS 接口和 AI 行为生成的验证结果	原型能验证核心流程和关键技术；AI 归纳结论需由开发人员复现或确认
S4 系统架构与详细设计	系统设计说明书、UML 模型、接口文档	AI 生成 UML 图形源码、接口草案、模块划分建议和异常路径说明	设计元素可追踪到需求；接口输入、输出和异常路径明确；设计评审通过
S5 编码实现与单元验证	源代码、单元测试报告、代码审查记录	AI 辅助局部代码生成、重构建议、测试桩和单元测试代码生成	开发人员能解释并修改 AI 代码；代码通过 Review、静态检查和单元测试
S6 集成测试与迭代修正	集成测试记录、缺陷记录、修正结果、回归测试记录	AI 辅助补充测试用例、边界场景、失败摘要和缺陷定位线索	缺陷可复现、可跟踪、可关闭；修复后通过回归测试；AI 测试建议经测试人员审核
S7 交付与确认	可运行系统、交付文档、验收材料、版本记录	AI 辅助生成发布说明、版本变更摘要和交付检查清单	代码、文档、测试报告和版本记录一致；交付材料经项目负责人确认
S8 项目总结与过程改进	项目总结报告、过程度量记录、改进建议	AI 生成日志摘要、缺陷归纳、复盘草稿和改进建议	AI 建议与人工确认结论区分清楚；复盘结论有事实依据并经团队确认
全过程	AI 使用记录、AI 输出审核记录、AI 采纳与修改记录	记录提示词、输入材料、输出内容、审核意见、采纳状态和修改情况	未审核 AI 输出不得进入正式交付物；AI 使用过程可追溯、可复盘

表5 交付物和质量标准表

3.7 过程度量指标

过程度量指标用于判断 AI 技术引入后是否真正改善了软件过程。本项目的度量指标分为三类：第一类是效率指标，用于观察 AI 是否缩短文档起草、测试设计和复盘整理时间；第二类是质量指标，用于观察 AI 输出是否真正提高了需求覆盖、测试覆盖和缺陷发现能力；第三类是过程记录指标，用于保证 AI 使用行为可追溯、可审核。

指标	类型	含义	记录阶段	用途
需求初稿生成耗时	效率指标	从准备需求材料到形成 SRS 初稿的时间	S2 需求获取与分析	对比人工方式与 AI 辅助方式的文档起草效率
设计草案准备耗时	效率指标	从准备需求和约束到形成 UML 草图、接口草案的时间	S4 系统架构与详细设计	观察 AI 对设计建模和接口草案准备的辅助效果
测试用例生成耗时	效率指标	从准备测试输入到形成测试用例初稿的时间	S5 编码实现与单元验证、S6 集成测试与迭代修正	观察 AI 对测试设计效率的影响
故障定位时间	效率指标	从发现缺陷到确认根因的时间	S6 集成测试与迭代修正、S8 项目总结与过程改进	观察 AI 日志摘要和失败结果分析是否有助于问题定位
AI 输出采纳率	质量指标	被采纳的 AI 输出数量 / AI 输出总数量	各阶段评审后	判断 AI 输出对项目过程的实际贡献
AI 输出返工率	质量指标	需要重写或大幅修改的 AI 输出数量 / AI 输出总数量	各阶段评审后	评估提示词质量和 AI 输出可靠性
需求覆盖率	质量指标	已被设计、代码或测试覆盖的需求数量 / 需覆盖需求数量	S2、S4、S6	判断需求是否在后续设计、实现和测试中得到追踪
自动化测试用例数量	质量指标	可执行自动化测试用例总数	S5、S6	衡量测试资产自动化程度
单元测试通过率	质量指标	通过的单元测试数 / 执行的单元测试数	S5 编码实现与单元验证	判断模块级验证结果是否满足进入集成测试的条件
缺陷发现数	质量指标	在测试和审查中发现的问题数量	S5、S6	观察 AI 辅助测试和审查对质量改进的影响
AI 使用记录完整率	过程记录指标	已完整记录的 AI 使用次数 / 实际 AI 使用次数	全过程	保证 AI 活动可追溯、可审核

表6 过程度量指标表

四、成果总结与未来展望

本次软件过程更新以第一次作业中定义的八阶段迭代式过程为基础，结合后续 AI 应用场景调研、软件过程改进方案设计和原型验证结果，对原有过程进行了 AI 增强。更新后的过程没有重新建立一套独立的 AI 开发流程，而是在项目启动与计划、需求获取与分析、原型设计与可行性验证、系统架构与详细设计、编码实现与单元验证、集成测试与迭代修正、交付与确认、项目总结与过程改进等阶段中，补充了 AI 辅助生成、AI 输出审核、AI 使用记录和过程度量等内容。

从本次更新结果看，AI 在本项目中更适合作为过程增强工具，而不是替代项目成员完成软件开发。它可以参与需求初稿、用户故事、UML 草图、接口草案、局部代码、测试用例、自动化测试脚本和复盘摘要等内容的生成，但这些输出都必须经过人工审核、质量标准检查和必要的测试验证，才能进入正式交付物或过程资产。通过这种方式，软件过程既能利用 AI 提高部分重复性工作的效率，也能保持原有角色职责和阶段准入机制的稳定。

后续如果继续完善该软件过程，可以从两个方向展开。一方面，可以继续补充更多验证场景，例如将 AI 辅助测试生成扩展到接口测试、集成测试和回归测试，并记录测试用例生成耗时、自动化测试用例数量、需求覆盖率、AI 输出采纳率和返工率等数据。另一方面，可以进一步沉淀过程资产，例如形成稳定的需求生成提示词模板、设计建模提示词模板、代码审查记录模板、AI 测试生成记录和复盘摘要模板，使 AI 使用不再是临时行为，而是能够被管理、被追踪、被复用的软件过程活动。

五、参考文献

[1] Capgemini Research Institute. *Turbocharging software with Gen AI: How organizations can realize the full potential of generative AI for software engineering*. 2024. <https://www.capgemini.com/insights/expert-perspectives/boosting-productivity-in-software-engineering-with-generative-ai>

[2] Stack Overflow. *2025 Developer Survey: AI*. 2025. <https://survey.stackoverflow.co/2025/ai>

[3] Google Cloud DORA. *Accelerate State of DevOps Report 2024*. 2024. <https://dora.dev/research/2024/dora-report/>

[4] Cheng H., Husen J. H., Lu Y., Racharak T., Yoshioka N., Ubayashi N., Washizaki H. *Generative AI for Requirements Engineering: A Systematic Literature Review*. arXiv, 2024/2025. <https://arxiv.org/abs/2409.06741>

[5] Mermaid. *Mermaid Documentation*. n.d. <https://mermaid.js.org/intro/>

[6] GitHub Docs. *Using GitHub Copilot code review*. n.d. <https://docs.github.com/en/copilot/using-github-copilot/code-review/using-copilot-code-review>

[7] GitHub Docs. *About code scanning with CodeQL*. n.d. <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning>

[8] SonarSource. *SonarQube Server Documentation*. n.d. <https://docs.sonarsource.com/sonarqube-server/>

[9] GitHub Blog. *Does GitHub Copilot improve code quality? Here's what the data says*. 2024. <https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/>

[10] ISO/IEC 12207:2017. *Systems and software engineering -- Software life cycle processes*. 2017.

[11] ISO/IEC 15504-5:2012. *Information technology -- Process assessment -- An exemplar software life cycle process assessment model*. 2012.

[12] ISO/IEC 33020:2019. *Information technology -- Process assessment -- Process measurement framework*. 2019.